

GAN-Based Robust Motion Planning for Mobile Robots Against Localization Attacks

Wenbing Tang¹, Yuan Zhou², *Member, IEEE*, Haiying Sun, Yuhong Zhang, Yang Liu³, *Senior Member, IEEE*,
Zuohua Ding⁴, *Associate Member, IEEE*, Jing Liu⁵, and Jifeng He

Abstract—Motion planning (MP) is essential but challenging for mobile robots. Most of the existing MP methods, at each instant, compute an action based on the states of the robot and the surrounding obstacles, assuming that the robot's localization module is attack-free. Unfortunately, the localization module is vulnerable to sensor attacks, such as GPS spoofing attacks. In this letter, we propose a novel robust framework, GAN-MP, where a generative adversarial network (GAN) is exploited to mitigate the localization attacks, and the state-of-the-art MP methods are applied to generate collision-free actions. Specifically, GAN-MP aims to learn a Generator to compute the potential positions of the robot. Consequently, it can reserve the robot's benign states while correcting the attacked states. Hence, it is suitable for benign and attacked scenarios without any attack detector. In addition, GAN-MP is method-agnostic and can be easily integrated with any existing MP method. We instantiate GAN-MP with a deep reinforcement learning method to demonstrate its design and training processes. Comprehensive experiments show that GAN-MP can mitigate localization attacks and guarantee safe motion. We also demonstrate the robustness and generalization of GAN-MP.

Index Terms—Generative Adversarial Networks, Localization Attacks, Robust Motion Planning.

Manuscript received 11 August 2022; accepted 16 January 2023. Date of publication 2 February 2023; date of current version 8 February 2023. This letter was recommended for publication by Associate Editor N. Karapetyan and Editor A. Bera upon evaluation of the reviewers' comments. This work was supported in part by National Key Research and Development under Grant 2019YFA0706404, in part by the National Nature Science Foundation of China under Grants 61972150 and 62132014, in part by the Fundamental Research Funds for Central Universities, in part by the Academic Research Fund Tier 2 Ministry of Education in Singapore under Grant MOE-T2EP20120-0004, and in part by the Zhejiang Provincial Key Research and the Development Program of China under Grant 2022C01045. (Corresponding authors: Yuan Zhou; Jing Liu.)

Wenbing Tang, Haiying Sun, Jing Liu, and Jifeng He are with the Shanghai Key Laboratory of Trustworthy Computing, East China Normal University, Shanghai 200062, China (e-mail: wenbingtang@hotmail.com; hysun@sei.ecnu.edu.cn; jliu@sei.ecnu.edu.cn; jifeng@sei.ecnu.edu.cn).

Yuan Zhou is with the School of Computer Science and Engineering, Nanyang Technological University, Singapore 639798 (e-mail: zhousyuan1130@163.com).

Yuhong Zhang is with Huawei Technology, Shanghai 201206, China (e-mail: john.zhangyh@huawei.com).

Yang Liu is with the School of Information Science and Technology, Zhejiang Sci-Tech University, Hangzhou 310018, China, and also with the School of Computer Science and Engineering, Nanyang Technological University, Singapore 639798 (e-mail: yangliu@ntu.edu.sg).

Zuohua Ding is with the School of Information Science and Technology, Zhejiang Sci-Tech University, Hangzhou 310018, China (e-mail: zouhuading@hotmail.com).

Digital Object Identifier 10.1109/LRA.2023.3241807

I. INTRODUCTION

MOTION planning (MP) aims to move a robot from its initial position to the target without causing collisions with obstacles and other robots [1]. Since it is one of the essential requirements for mobile robots, many MP methods have been proposed, such as sampling-based methods [2], mathematical programming [3], velocity obstacles [4], [5], [6], and deep reinforcement learning (DRL) [7], [8], [9]. Generally, they take the robot's state (i.e., the position and velocity) and the surrounding obstacles' states as input and generate a collision-free action. Especially, they always assume that the robot can localize itself exactly using different sensors, such as Global Positioning System (GPS), Inertial Measurement Unit (IMU), camera, and Light Detection and Ranging (LiDAR).

However, current research shows that robots are vulnerable to various localization attacks (LAs), such as GPS spoofing attacks [10], IMU injection attacks [11], and LiDAR replay attacks [12]. The attacks will generate wrong sensor measurements, resulting in an incorrect estimation of the robot's position. When an MP controller receives the fake state of the robot, it generates a wrong action [12], causing severe consequences, e.g., collisions and even deaths. Existing work mainly focuses on attack detection and mitigation [12], [13], [14]. There are two challenges with the current methods. First, attack detection is the essential prerequisite for attack mitigation. However, the current detectors can only detect some specific attacks, and there are no unified and perfect detectors to detect all kinds of attacks. Second, the current mitigation methods are cost- and time-consuming, and even impossible to address real-time attacks in practice.

In this paper, focusing on their detrimental effects, we propose a general detector-agnostic robust framework, GAN-MP, to mitigate LAs against a mobile robot. On the one hand, GAN-MP can deal with benign and attack scenarios, with no detectors required. On the other hand, GAN-MP is attack-agnostic and can directly generate the robot's approximate, if not accurate, positions via a well-trained generative adversarial network (GAN). So, it can mitigate various LAs in real time. Remarkably, GAN-MP is also method-agnostic and can be integrated with any MP method. In this paper, we instantiate GAN-MP with a DRL method. Specifically, we implement it with two modules: a GAN-based module to correct the robot's attacked positions and a DRL module to compute collision-free actions. First, we train a GAN model to correct the attacked data, where the well-trained Generator

can reserve the robot's benign positions while predicting the potential values of the attacked ones. Second, the corrected robot state and the encoded obstacles' state are fed to the value network of DRL to compute a collision-free action.

We conduct comprehensive experiments to evaluate the effectiveness and efficiency of the implemented GAN-MP method. First, we perform the method on various scenarios with a variable number of obstacles and four LAs. The results show that GAN-MP can mitigate different LAs and guarantee collision avoidance. Second, we compare our GAN-based mitigation strategy with other attack-resilience methods, and the results show the robustness of our approach. Third, we integrate the GAN-based mitigation strategy into other MP methods to demonstrate the generalization of GAN-MP.

The contributions of this paper are:

- We propose the first GAN-based strategy to mitigate LAs in mobile robots without any attack detector.
- We propose a robust framework GAN-MP and instantiate a method GAN-DRL, where a DRL method is selected for motion planning of robots moving around in an environment with a variable number of obstacles and LAs.
- We conduct extensive experiments to demonstrate the effectiveness and efficiency of GAN-MP.

II. RELATED WORK

This paper is related to the area of MP, which is an important problem in robotics. Many methods have been proposed in this area. They can be mainly classified into two categories: conventional methods and DRL-based methods. The conventional methods, such as velocity obstacles [4], [5], [6], rely on the precise models of robots and/or the environment. However, due to the lack of self-learning ability, they are hard to be applied in various environments. In addition, most conventional MP methods cannot guarantee computation efficiency and smooth motion simultaneously [4], [15].

Since there is a better trade-off between smooth motion and computation efficiency [8], DRL has been applied in MP. Based on the inputs of a DRL model, we divide the current DRL models into two categories: sensor-based DRL models [16], [17], which take the raw data from sensors as inputs, and agent-based DRL models [7], [8], which take the agent information (e.g., types and states) as inputs. In practice, with agent-level information from multiple sensors, the DRL models can make more precise decisions [18]. However, one challenge for agent-based DRL models is the variable number of obstacles. To handle a variable number of obstacles, the authors in [19] proposed a DRL method with a long short-term memory (LSTM) model, where the LSTM model is used to transform the obstacles' states into a fixed-size hidden state based on their distances to the robot. Inspired by [19], Xu et al. [9] proposed a novel LSTM model to encode a variable number of obstacles, where the states of obstacles are ordered based on their collision risks with the robot. However, these methods assume that the robot's state obtained from the localization module is always reliable, which is inappropriate for today's adversarial environments containing various LAs. Hence, in this paper, we propose a robust MP method for robots moving around in adversarial environments.

In the case of LAs, some attack resilience techniques have been proposed. However, the current methods always require detecting the attacks in advance, which is still a significant challenge. For example, the software sensor-based recovery (SSR) technology [13] generates software sensors, i.e., approximate models of the corresponding physical sensors, to replace the corresponding attacked ones using system identification techniques. In [20], an LSTM model is applied to correct attacked sensors' data and construct an attack-resilient controller running concomitantly with the primary PID controller; the backup controller will replace the primary one when attacks against sensors are detected. Different from these works, our method does not need any attack detector.

III. BACKGROUND AND PROBLEM STATEMENT

A. Motion Planning

In this paper, we consider the MP problem for a holonomic robot moving from its initial position $\mathbf{p}_i = [x_i, y_i]$ towards its target position $\mathbf{p}_g = [x_g, y_g]$ in an unknown environment with obstacles (e.g., pedestrians and other robots). Suppose the safety radius of the robot is r . The continuous motion time can be divided into a set of discrete time instants with an equal time step Δt . At each time instant $t, t \in \{0, 1, 2, \dots\}$, the robot computes a collision-free velocity for the duration $[t\Delta t, (t+1)\Delta t)$. Hence, the robot's state and action at t can be represented as $\mathbf{s}_t = [\mathbf{p}_t, \mathbf{v}_t^-, \mathbf{p}_g, v_f, r] \in \mathbb{R}^8$ and $\mathbf{v}_t$, respectively, where $\mathbf{p}_t = [x_t, y_t]$ is the robot's position at t , $\mathbf{v}_t^-$, satisfying $\mathbf{v}_t^- = \mathbf{v}_{t-1} = [vx_{t-1}, vy_{t-1}]$ for $t \geq 1$ and $\mathbf{v}_0^- = [0, 0]$, is the robot's velocity in the duration $[(t-1)\Delta t, t\Delta t)$, v_f is the preferred speed, and $\mathbf{v}_t$ is the velocity that should be determined at t . Suppose the detected obstacles at t are $\{o_t^1, \dots, o_t^n\}$, whose states are denoted as $\mathbf{S}_t = \{\mathbf{so}_t^1, \dots, \mathbf{so}_t^n\}$, where $\mathbf{so}_t^j = [\mathbf{p}_t^j, \mathbf{v}_t^j, r_j] \in \mathbb{R}^5$, and $\mathbf{p}_t^j = [x_t^j, y_t^j]$, $\mathbf{v}_t^j = [vx_t^j, vy_t^j]$, and r_j are the position, velocity, and radius of o_t^j , respectively. Hence, our MP problem is to determine a motion policy $\pi : (\mathbf{s}_t, \mathbf{S}_t) \mapsto \mathbf{v}_t$, mapping state information to collision-free actions, such that the robot can arrive at the target as soon as possible without any collision. It can be formally described as:

$$\arg \min_{\pi} T \quad (1)$$

$$s.t. \quad \mathbf{p}_0 = \mathbf{p}_i, \mathbf{p}_T = \mathbf{p}_g, \quad (2)$$

$$\|\mathbf{p}_t - \mathbf{p}_t^j\| \geq r + r_j, j \in \{0, 1, \dots\}, \forall t, \quad (3)$$

$$\mathbf{p}_{t+1} = \mathbf{p}_t + \mathbf{v}_t \Delta t, \forall t \in \{0, 1, \dots, T-1\}, \quad (4)$$

where the motion time T in (1) is the optimization objective, (2) is the position constraints, (3) is the safety (collision avoidance) constraints, and (4) is the kinematics of the robot. Existing MP methods try to generate better solutions to the optimization problem from different perspectives. According to [8], [19], such an optimization problem can be solved efficiently in the DRL framework:

$$\mathbf{v}_t^* = \arg \max_{\mathbf{v} \in \mathcal{A}} R(\mathbf{s}_t, \mathbf{S}_t, \mathbf{v}) + \gamma^{\Delta t \cdot v_f} V^*(\mathbf{s}_{t+1, \mathbf{v}}, \mathbf{S}_{t+1}), \quad (5)$$

where $\mathcal{A}$ is the set of candidate actions, $R(\mathbf{s}_t, \mathbf{S}_t, \mathbf{v})$ is the one-step reward by performing action $\mathbf{v}$ at $[\mathbf{s}_t, \mathbf{S}_t]$ (the corresponding

next state is $s_{t+1,v}$, $\gamma \in [0, 1]$ is a discount factor, $V^*(\cdot)$ is the optimal value function. The reward function R can be found in [9] and [7]. Note that even though our descriptions focus on 2D environments, this framework can be also applied to 3D environments.

B. Localization Attacks

a) Attack Goal: A robot needs to localize itself during its motion based on the data from different sensors, such as GPS, IMU, LiDAR, and odometry. The localization results are used by the robot's planning and control modules and affect their performance significantly. Even though some sensor fusion techniques, e.g., Kalman Filter and particle filter, have been proposed to produce more accurate state estimation [21], the robots may still suffer from LAs [22]. In this paper, the attack goal is to modify the robot's actual position and generate a fake position. When the MP controller receives the wrong position, it makes a wrong decision.

b) Threat Model: To achieve the above attack goal, we assume that LAs are performed by faking the robot's positions. Formally, given the robot's current position $\mathbf{p}_t$, LAs will generate a faked position $\tilde{\mathbf{p}}_t$ such that $\tilde{\mathbf{p}}_t = \alpha \odot \mathbf{p}_t$, where α is a scalar or a vector with proper dimensions, and $\odot$ can be any proper operator (e.g., multiplication or addition) that returns a vector with the same dimension of $\mathbf{p}_t$. For example, at time t , an attacker may tamper the robot's real position $[x_t^r, y_t^r]$ by multiplying it with a scalar α , resulting in a fake location $[\alpha x_t^r, \alpha y_t^r]$. Hence, a robust planning algorithm needs to disentangle $[x_t^r, y_t^r]$ from the attacked one. Note that LAs are significantly different from the perturbations of real locations. An attacker can perform any LA according to the attack intention and modify the positions significantly, which is independent of the current real locations. However, the locations generated by perturbations are usually close to the current measured locations [23]. We may estimate the maximal perturbations based on the sensors' specifications. Additionally, we assume the attacker cannot access the robots' non-position states, e.g., the preferred speed and the target position, and cannot poison the proposed GAN-MP.

C. Generative Adversarial Networks

A GAN model is typically composed of a Generator and a Discriminator. The Generator produces synthetic samples with the same latent distribution as the training data, and the Discriminator is to classify the samples as real (from the training data) or fake (generated by the Generator). In practice, the Generator and the Discriminator are two neural networks trained simultaneously in a zero-sum game framework. The training process of a GAN model aims to update two neural networks such that the Generator can fool the Discriminator effectively [24]. GANs have been introduced to generate new realistic samples in many applications [25], such as image generation, language and speech synthesis, and image-to-image translation. In this paper, we apply GANs to generate the potential positions for a robot under LAs.

Hence, our problem can be formulated as: *Given a robot moving around in an unknown environment under LAs, design*

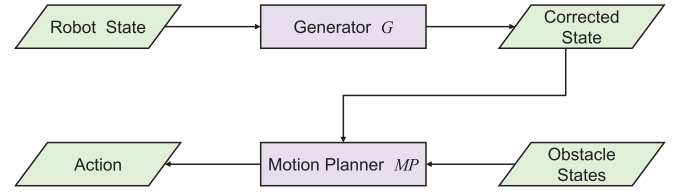


Fig. 1. The general framework of GAN-MP.

a robust motion planning method such that the robot can safely move towards the target position without any attack detector.

IV. APPROACH

In this section, we present the design of our robust motion planning framework GAN-MP and illustrate the training process with an instantiation GAN-DRL.

A. Robust Motion Planning Framework GAN-MP

The general framework of GAN-MP is shown in Fig. 1. It contains two essential modules: a GAN-based module for LAs mitigation and an MP module for action generation. Hence, GAN-MP can be formally described as $\text{GAN-MP} := \langle GAN, MP \rangle$, where GAN represents the Generator model for a well-trained GAN model, and MP represents any motion planning method. The Generator will output the corrected state of the robot, and the MP module will take the corrected states and the states of obstacles as inputs and computes the corresponding motion commands. In detail, at each time instant t , the robot's position $\mathbf{p}_t$ is first corrected via the Generator. Therefore, the corrected position $\mathbf{p}_t^G$ will be concatenated with the robot's non-position states $\mathbf{s}_t \setminus \mathbf{p}_t$ to obtain the corrected state $\mathbf{s}_t^G$. Then, the robot's corrected state $\mathbf{s}_t^G$ and the obstacles' states $\mathbf{S}_t$ will be fed to the motion planner to generate an action $\mathbf{v}_t$. Next, we give the details of these two modules.

1) GAN-Based Mitigation Strategy: To correct the robot's attacked positions, we train a GAN model to generate the potential positions of the robot under LAs. The main training purpose is to guide the Generator to generate an approximate position of the real one. The detailed model is given as follows.

a) Generator: The Generator G is a multi-layer perceptron (MLP). At any time instant t , its input is:

$$z_t = [\mathbf{p}_{t-1}, \mathbf{v}_{t-1}, \mathbf{p}_t^m] \in \mathbb{R}^6 \quad (6)$$

where $\mathbf{p}_{t-1} = [x_{t-1}, y_{t-1}]$ and $\mathbf{v}_{t-1} = [vx_{t-1}, vy_{t-1}]$ are the robot's recorded position and velocity at $t-1$, and $\mathbf{p}_t^m = [x_t^m, y_t^m]$ is the robot's current measured position, which may be poisoned by an attacker. The output of G is the robot's potential position $\mathbf{p}_t^G = [x_t^G, y_t^G]$. Hence, we have

$$\mathbf{p}_t^G = G(z_t; \theta_G) \quad (7)$$

where θ_G are parameters of G that need to be learnt.

b) Discriminator: The Discriminator D is another MLP. It takes the real data $l_{real} = [\mathbf{p}_{t-1}, \mathbf{v}_{t-1}, \mathbf{p}_t]$ or the generated data $l_{gen} = [\mathbf{p}_{t-1}, \mathbf{v}_{t-1}, \mathbf{p}_t^G]$ as the input, and outputs the classification result $D(l; \theta_D)$, where $l = l_{real}$ or $l = l_{gen}$, and θ_D are the parameters of D . Hence, the objective function for the

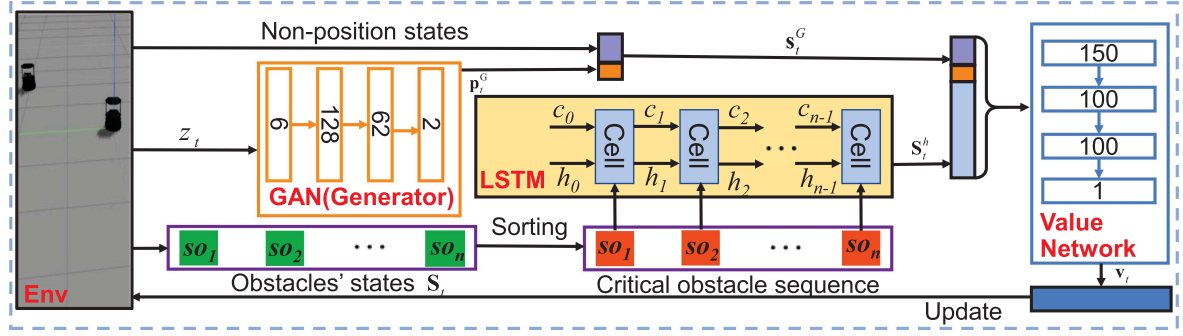


Fig. 2. The architecture of GAN-DRL.

training of the GAN model can be described as:

$$\min_{\theta_G} \max_{\theta_D} F(\theta_D, \theta_G) = \mathbb{E}_{l_{real}} [\log D(l_{real})] + \mathbb{E}_{z_t} [\log(1 - D(l_{gen}))] \quad (8)$$

2) *Motion Planning Module*: When the GAN model is trained, we can use the corresponding Generator G to generate the robot's potential positions, which can correct the attacked positions while reserving the benign ones. Therefore, the Generator can be integrated with any MP method to build a compound robust planning algorithm. As shown in Fig. 1, replacing the *MP* module with a concrete planning method can generate an instantiation of GAN-MP. In the sequel, taking the DRL-based motion planning method as an example, we demonstrate the training process of GAN-MP. More experiments of instantiating GAN-MP with different motion planning methods will give in Section V-D.

B. Training Process of GAN-MP

In this section, we illustrate the training process of GAN-MP by instantiating the *MP* with a DRL method proposed in [9]. The resulting method is denoted as GAN-DRL. As shown in Fig. 2, GAN-DRL contains three sub-models: a Generator model, an LSTM model, and a value network, where the LSTM model is used to deal with a variable number of obstacles in the environment. The training process contains two stages. The first one is training a GAN model on the training dataset $L = \{[p_{t-1}, v_{t-1}, p_t^m]\}$. Note that during the training phase of the GAN model, the previous position p_{t-1} in L is attack-free. However, in the inference phase, p_{t-1} is the output of the Generator at $t-1$. The second one is training a DRL model (i.e., the LSTM model and the value network) according to $s_t^G = [p_t^G, v_t^-, p_g, v_f, r]$ and S_t , where p_t^G is generated by the Generator trained at the first stage. If a conventional method is selected as *MP* in GAN-MP, no training process is required for the second stage.

The training process is described in Algorithm 1. Lines 1 and 2 describe the training of a GAN model, where the training dataset is generated from ORCA [5]. Lines 3-31 are the training process for the DRL model. First, the value network is initialized based on a set of trajectories generated from ORCA (Lines 3 and 4). Second, the replay memory E and the local target value

Algorithm 1: Training Process for GAN-DRL.

Input: Training episodes M , replay iterations N , the action space $\mathcal{A}$, the exploration probability ϵ , the maximal time steps T_m , and the update frequency for the target value network ω .

Output: Trained GAN-DRL.

```

1: Collect a training dataset  $L$  via ORCA;
2: Train a GAN on  $L$  by optimizing (8);
3: Collect a set of trajectories via ORCA;
4: Train an initial value network  $V$  on the trajectories;
5: Initialize a replay memory  $E$  from the trajectories;
6: Initialize the target network  $\hat{V} \leftarrow V$ ;
7: for episode = 1 :  $M$  do
8:   for ite = 1 :  $N$  do
9:     Sample  $s_0, S_0$ , and  $p_g$ ;
10:    done = False,  $t = 0$ ,  $Traj = \{[s_0, S_0]\}$ ;
11:    while not done do
12:      Detect states  $s_t$  and  $S_t$ ;
13:       $S_{t+1}^h \leftarrow LSTM(criticality(S_{t+1}))$ ;
14:      Generate  $s_t^G$  based on the Generator and  $s_{t-1}^G$ ;
15:      Select  $v_t$  from  $\mathcal{A}$  using  $\hat{V}$  and the  $\epsilon$ -greedy policy;
16:      The robot moves to  $s_{t+1}$  controlled by  $v_t$ ;
17:      Detect the new obstacles' states  $S_{t+1}$ ;
18:       $t \leftarrow t + 1$ ,  $Traj \leftarrow Traj \cup \{[s_{t+1}, S_{t+1}]\}$ ;
19:      if collided or reached or  $t > T_m$  then
20:        done = True;
21:      end if
22:    end while
23:    if collided or reached then
24:      update the memory  $E$  with  $Traj$ ;
25:    end if
26:  end for
27:  Update  $V$  and  $LSTM$  by gradient descent with  $E$ ;
28:  if (episode mod  $\omega$ ) == 0 then
29:    Update target network  $\hat{V} \leftarrow V$ ;
30:  end if
31: end for

```

network $\hat{V}$ are initialized (Lines 5 and 6). Third, the algorithm repeats M episodes to train the DRL model. We replay the robot motion N times for each episode and then update the memory E . In each replay, we reset the initial state $[s_0, S_0]$ and the target

position $\mathbf{p}_g$ randomly, and then control the robot to move to $\mathbf{p}_g$ (Lines 11-22). At any instant t , the algorithm predicts $\mathbf{S}_{t+1}$ and computes the hidden state $\mathbf{S}_{t+1}^h$ via the Crit-LSTM given in [9] (Line 13), where the function $\text{criticality}(\cdot)$ sorts the input based on the risks of the obstacles to the robot. The ϵ -greedy policy is used to select $\mathbf{v}_t$ (Line 15), which can be described as:

$$\mathbf{v}_t = \begin{cases} \text{randomly select from } \mathcal{A}, & \text{with probability } \epsilon; \\ \arg \max_{\mathbf{v} \in \mathcal{A}} Q(\mathbf{s}_t^g, \mathbf{S}_t, \mathbf{v}), & \text{with probability } 1 - \epsilon; \end{cases}$$

where $Q(\mathbf{s}_t^g, \mathbf{S}_t, \mathbf{v}) = R(\mathbf{s}_t^g, \mathbf{S}_t, \mathbf{v}) + \gamma^{\Delta t \cdot v_f} \hat{V}(\mathbf{s}_{t+1}^g, \mathbf{S}_{t+1}^h)$ and $\mathbf{s}_{t+1}^g = \mathbf{s}_t^g + \mathbf{v} \Delta t$. The robot will move with $\mathbf{v}_t$ in the duration $[t\Delta t, (t+1)\Delta t)$. When the robot collides with an obstacle, arrives at its target position, or reaches the maximal motion time, the current replay is terminated (Lines 19-21). Only when the replay is terminated with the collided or reached status, the generated trajectory is used to update the replay memory E (Lines 23-25). At the end of each episode, the weights of value network V and LSTM are updated simultaneously by gradient descent. The local target network $\hat{V}$ is updated every ω episodes (Lines 28-30).

V. EVALUATION

In this section, we first evaluate the performance of GAN-DRL and then the generalization of GAN-MP. We conduct our experiments in the simulation environment developed by [7]. For the training of the GAN model, we collect 50,000 samples via ORCA. The network architectures of the Generator and the Discriminator are (6,128,64,2) and (6,128,64,1), respectively. We train the Generator and the Discriminator iteratively for 10 epochs, with a batch size of 64. Their initial learning rates are 0.001. In the training phase of the Generator, to simulate LAs, the attacked position $\mathbf{p}_t^m$ in each z_t is generated from the Gaussian distribution $N(0, 1)$.

For the training of the DRL model, the preferred speed $v_f = 1$, and the action space $\mathcal{A} = \{(v_i \cos \theta_j, v_i \sin \theta_j) | v_i = (e^{i/5} - 1)/(e - 1), \theta_j = j/8, i = 1, \dots, 5, j = 0, \dots, 15\}$. In each replay, all obstacles are randomly located on the circumference of a circle and need to move to the opposite side of the circle. In addition, $N = 1, \omega = 50, T_m = 100$, the dimension of the LSTM's hidden state is 50, the values of c_0 and h_0 are zero vectors, and the architecture of the value network is (150,100,100,1). The DRL model is trained for 30,000 episodes with a variable number of obstacles, varying from 1 to 10. All models are implemented in PyTorch.

A. Overall Performance of GAN-DRL

To evaluate the effectiveness of GAN-DRL, we compare GAN-DRL with the corresponding pure DRL method (denoted as pDRL). Note that pDRL is trained with the same parameters in the DRL training stage of GAN-DRL. We evaluate either model under two situations: with LAs and without LAs. In the attacked scenario, the robot is under spoofing attacks from the second time step to the termination of the motion.

We conduct 10 independent test sets for GAN-DRL and pDRL. Each test set contains 500 test cases, where the number of

obstacles varies from 1 to 10. The measures to evaluate the experiments are *success rate*, *collision rate*, *timeout rate*, *average navigation time of the successful runs (NavTime)*, and *average decision time of the successful runs (DecTime)*. Table I shows their means and standard deviations (SDs) over 10 test sets. From Table I, we have the following conclusions:

- *The Generator does not affect the performance of the DRL model.* The table shows that the two models show almost the same performance on the five metrics in the attack-free scenario. It means that the GAN-based mitigation strategy can reverse the robot's benign states.
- *GAN-DRL can mitigate LAs significantly.* In the case of LAs, GAN-DRL can obtain a 96.88% success rate, while the success rate of pDRL is only 83.88%. pDRL causes a higher average collision rate (14.78% vs. 1.62%), which means the robot falls into collisions at many runs under LAs.
- *GAN-DRL can maintain smooth motion.* Under attacks, the average navigation time of pDRL increases (from 11.351s to 13.894s). In contrast, GAN-DRL can guarantee smooth motion of the robot as the average navigation time (*NavTime*) is similar in the attack-free and the attacked scenarios (11.676s for the attacked scenario and 11.388s for the attack-free one).
- *GAN-DRL does not affect the real-time performance.* From the table, we can find that compared with pDRL, GAN-DRL has similar average decision time (*DecTime*) in either scenario.

Hence, we can conclude that GAN-DRL can mitigate LAs efficiently without causing performance degradation. Moreover, as GAN-DRL is suitable for benign and attacked scenarios, no attack detectors are required.

In the sequel, we further show some trajectories computed by the two methods with and without LAs, as shown in Fig. 3. From Fig. 3(a), we can find that in the benign scenario, pDRL can control the robot to move towards its target smoothly. As shown in Fig. 3(b), GAN-DRL can also generate a smooth trajectory for the robot, which shows that the GAN-based mitigation strategy can reserve benign states. Fig. 3(c) shows that under the current LA, the robot collides with *obstacle1* in the 19-th time interval and fails to reach its target, as pDRL generates a wrong action according to the faked positions. In addition, the robot's trajectory oscillates in the attacked scenario. From Fig. 3(d), we can see that using GAN to correct attacked positions can navigate the robot to its target while maintaining smooth motion. The comparison between Fig. 3(c) and (d) illustrates that GAN-DRL can correct attacked states and navigate the robot towards its target safely and smoothly. The comparison between Fig. 3(b) and (d) illustrates that GAN-DRL can deal with both benign and attacked scenarios using a unified GAN-based mitigation strategy without any attack detector.

B. Performance of GAN-DRL Under Different Attacks

This section evaluates the mitigation capacity of GAN-DRL under different LAs. Inspired by [22], we design four types of LAs: (1) *Total Failure*, where all localization sensor signals are

TABLE I
PERFORMANCE COMPARISON BETWEEN GAN-DRL AND pDRL IN THE ATTACK-FREE AND THE ATTACKED SCENARIOS

Attack	Method	Success		Collision		Timeout		NavTime (s)		DecTime (s)	
		Mean	SD	Mean	SD	Mean	SD	Mean	SD	Mean	SD
N	pDRL	0.982	1.549E-2	0.003	3.795E-3	0.015	1.212E-2	11.351	1.461E+0	0.110	4.074E-2
	GAN-DRL	0.983	1.479E-2	0.003	5.181E-3	0.014	1.069E-2	11.388	1.500E+0	0.110	4.090E-2
Y	pDRL	0.839	6.846E-2	0.148	6.050E-2	0.013	9.240E-3	13.894	1.594E-0	0.108	4.165E-2
	GAN-DRL	0.969	3.160E-2	0.016	2.147E-2	0.015	1.096E-2	11.676	2.541E+0	0.110	4.121E-2

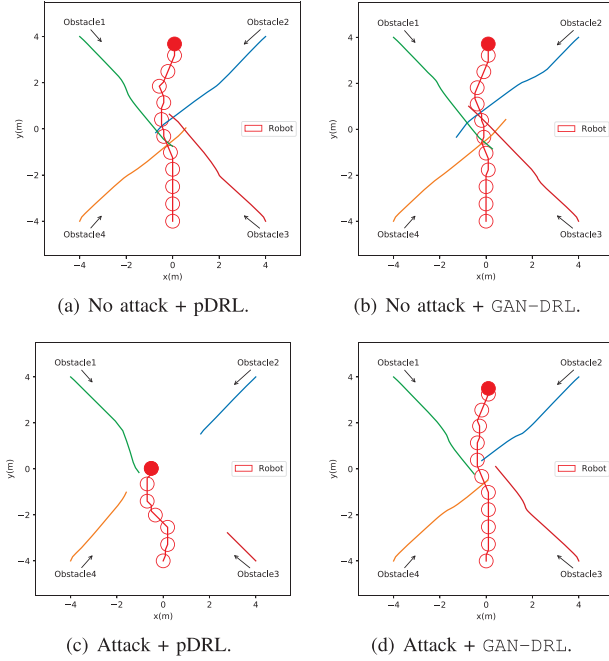


Fig. 3. The trajectories of the robot and four obstacles. Circles are the locations of the robot. The target of the robot is $[0, 4]$. The solid line without circles represents the trajectories of the obstacles. Their target positions are $[4, -4]$, $[-4, -4]$, $[-4, 4]$, and $[4, 4]$, respectively. All obstacles are controlled by ORCA. The attack is biased spoofing, i.e., adding a uniform distribution $U(0, 1.2)$ to the real values.

TABLE II
SUCCESS RATES AND SIGNIFICANCE TESTING OF GAN-DRL AND pDRL UNDER FOUR LOCALIZATION ATTACKS

Type	Method	Mean	SD	Levene's Test	T-Test
1	pDRL	0.702	0.120	0.015	1.493E-7
	GAN-DRL	0.939	0.056		
2	pDRL	0.803	0.075	0.251	2.000E-6
	GAN-DRL	0.923	0.068		
3	pDRL	0.796	0.129	0.005	2.839E-3
	GAN-DRL	0.941	0.049		
4	pDRL	0.816	0.114	0.026	7.110E-4
	GAN-DRL	0.946	0.051		

zero; (2) *Biased Signal*, where the sensor signals are added with a uniform distribution $U(0, 1.2)$; (3) *Multiplicative*, where the sensor signals are multiplied by 1.5; (4) *Replacement*, where the robot's positions are replaced by a value in $[-1, 1]$. We perform 15 independent test sets for each attack. Each test set contains 500 test cases with a variable number of obstacles, from 1 to 15. For comparison, we also perform all test cases using pDRL. The average success rate and the SD of either method are given in Table II. The results show that GAN-DRL

TABLE III
PERFORMANCE COMPARISON AMONG DIFFERENT DEFENSE STRATEGIES

Strategy	Success	Collision	Timeout	Correction Error
Kalman Filter	0.573	0.173	0.254	0.015
SSR	0.755	0.061	0.184	0.009
PID-Piper	0.640	0.344	0.016	0.011
GAN	0.779	0.174	0.047	0.008

can achieve a higher average success rate and a lower SD under each attack. To further evaluate the statistical significance of the results, we conduct Levene's test for equal variances and T-test for equal means. The result is statistically significant, which means there is a significant difference in the success rate between GAN-DRL and pDRL. Hence, we can conclude that **GAN-DRL can mitigate different LAs efficiently and guarantee a higher success rate.**

C. Comparison With Other Defense Strategies

We further compare our GAN-based strategy with other strategies in the literature. We select three recovery strategies: Kalman Filter [26], SSR [13] and PID-Piper [20]. In detail, Kalman Filter combines the sensor readings and the predicted positions to provide more accurate position estimations. SSR aims to learn surrogate models (i.e., discrete-time state-space models) to replace the attacked sensors and generate the position data from the model for the motion controller. PID-Piper applies an LSTM model to predict the attacked sensors' data for the motion controller. In our experiments, we implement SSR for the attacked position sensor using Matlab's System Identification Toolbox. We also use the generated state-space model of SSR as the dynamics in the prediction stage of Kalman Filter. The size of the hidden state in the LSTM model of PID-Piper is 5. For a fair comparison, their motion controllers are pDRL. To evaluate the robustness of the strategies, we add noise from the Gaussian distribution $N(0, 1)$ to the training and testing datasets. Other settings are the same as those given in Section V-A.

We first compute the average success/collision/timeout rates to evaluate the attack mitigation capability of four strategies. We also compute the correction error (i.e., mean absolute error between the actual position $\mathbf{p}_t$ and the corrected position $\mathbf{p}_t^G$) to evaluate the position correction capability. The results are given in Table III. The results show that the GAN-based mitigation strategy performs better than the other three. The reason is that by precisely correcting the attacked positions via the GAN-based mitigation strategy, the DRL model can generate better motion commands. Therefore, **the proposed GAN-based mitigation strategy can perform more accurate position corrections and deal with regular disturbances in the data.**

TABLE IV
PERFORMANCE OF THE GAN-BASED MITIGATION STRATEGY WITH DIFFERENT
MOTION PLANNING METHODS

Attack	Model	Success	Collision	Timeout
N	LSTM-RL	0.817	0.097	0.086
	GAN-LSTM-RL	0.817	0.097	0.086
Y	LSTM-RL	0.699	0.257	0.044
	GAN-LSTM-RL	0.838	0.115	0.047
N	SARL	0.992	0.006	0.002
	GAN-SARL	0.991	0.006	0.003
Y	SARL	0.790	0.207	0.003
	GAN-SARL	0.958	0.040	0.002
N	ORCA	0.999	0.000	0.001
	GAN-ORCA	0.999	0.000	0.001
Y	ORCA	0.997	0.000	0.003
	GAN-ORCA	0.999	0.000	0.001

D. Generalization of GAN-MP

Finally, we evaluate the generalization of GAN-MP, i.e., the compatibility of the GAN-based mitigation strategy with other MP methods. We select three well-established MP methods: LSTM-RL [19], SARL [7], and ORCA [5]. The first two are learning-based methods, while the last one is a conventional collision avoidance method widely applied in practice. LSTM-RL and SARL are also trained for 30,000 episodes. Hence, we have six models: LSTM-RL, GAN-LSTM-RL, SARL, GAN-SARL, ORCA, and GAN-ORCA. Similarly, we conduct 10 independent test sets for each model. Each test set contains 500 test cases, where the number of obstacles varies from 1 to 10. We evaluate all six models under two situations: with LAs and without LAs. Note that the attack implemented here is the same as in Section V-A. Table IV shows the average success/collision/timeout rates over the 10 test sets.

First, like GAN-DRL, the three GAN-based models show similar performance with their corresponding GAN-free models under the attack-free scenario. Second, under the attacked scenario, the two GAN-free learning-based models (LSTM-RL and SARL) show a significant decrease in the success rate and an increase in the collision rate. In contrast, the GAN-based models maintain a high success rate and low collision and timeout rates. Moreover, compared with pDRL and SARL, LSTM-RL shows a lower success rate and a higher collision rate. The reason is that LSTM-RL only considers the distance of an obstacle to the robot. Therefore, it cannot affect the real influence of the obstacle on the robot. Third, ORCA shows a relatively stable performance. It means ORCA is more robust than the learning-based methods.

In conclusion, **the proposed GAN-based mitigation strategy against LAs is universal and can be applied with different MP methods.** Moreover, ORCA can obtain high and stable performance, but its computation time increases exponentially with the number of robots [6], which limits its application in real-time systems. DRL-based methods can achieve high computation efficiency but lack robustness. Hence, we should pay more attention to promoting the robustness of learning-based MP methods.

VI. CONCLUSION

In this letter, we propose the first GAN-based mitigation strategy for localization attacks against mobile robots. By training a

GAN model, we apply its Generator to approximate the actual values of the attacked positions while reserving the benign ones without detectors. Then, we propose a robust motion planning framework GAN-MP to improve the robustness of motion planning methods in adversarial environments. We also instantiate a method GAN-DRL to address a variable number of obstacles in an environment with localization attacks. The comprehensive simulation experiments demonstrate the effectiveness and efficiency of the proposed approach. Our future work will focus on obtaining more effective and robust motion planning methods. We also plan to implement and test our approach on real-world robot platforms.

REFERENCES

- [1] W. Shi, Y. Zhou, X. Zeng, S. Li, and M. Bennewitz, "Enhanced spatial attention graph for motion planning in crowded, partially observable environments," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2022, pp. 4750–4756.
- [2] H. Nichols, M. Jimenez, Z. Goddard, M. Sparapany, B. Boots, and A. Mazumdar, "Adversarial sampling-based motion planning," *IEEE Robot. Automat. Lett.*, vol. 7, no. 2, pp. 4267–4274, Apr. 2022.
- [3] Y. Zhou, H. Hu, Y. Liu, S.-W. Lin, and Z. Ding, "A real-time and fully distributed approach to motion planning for multirobot systems," *IEEE Trans. Syst. Man Cybern.: Syst.*, vol. 49, no. 12, pp. 2636–2650, Dec. 2019.
- [4] J. A. Douthwaite, S. Zhao, and L. S. Mihaylova, "Velocity obstacle approaches for multi-agent collision avoidance," *Unmanned Syst.*, vol. 7, no. 01, pp. 55–64, 2019.
- [5] J. Van Den, S. J. Berg, M. Guy Lin, and D. Manocha, "Reciprocal n-body collision avoidance," in *Proc. Robot. Res.: 14th Int. Symp.*, 2011, pp. 3–19.
- [6] W. Tang, Y. Zhou, T. Zhang, Y. Liu, J. Liu, and Z. Ding, "Cooperative collision avoidance in multirobot systems using fuzzy rules and velocity obstacles," *Robotica*, vol. 41, no. 2, pp. 668–689, 2022.
- [7] C. Chen, Y. Liu, S. Kreiss, and A. Alahi, "Crowd-robot interaction: Crowd-aware robot navigation with attention-based deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2019, pp. 6015–6022.
- [8] Y. F. Chen, M. Liu, M. Everett, and J. P. How, "Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2017, pp. 285–292.
- [9] L. Xu, F. Wu, Y. Zhou, H. Hu, Z. Ding, and Y. Liu, "Criticality-guided deep reinforcement learning for motion planning," in *Proc. China Automat. Congr.*, 2021, pp. 3378–3383.
- [10] K. C. Zeng et al., "All your GPS are belong to us: Towards stealthy manipulation of road navigation systems," in *Proc. USENIX Secur.*, 2018, pp. 1527–1544.
- [11] T. Trippel, O. Weisse, W. Xu, P. Honeyman, and K. Fu, "WALNUT: Waging doubt on the integrity of MEMS accelerometers with acoustic injection attacks," in *Proc. IEEE Secur. Privacy*, 2017, pp. 3–18.
- [12] Q. Liu, Y. Mo, X. Mo, C. Lv, E. Mihankhah, and D. Wang, "Secure pose estimation for autonomous vehicles under cyber attacks," in *IEEE Trans. Intell. Veh.*, 2019, pp. 1583–1588.
- [13] H. Choi, S. Kate, Y. Aafer, X. Zhang, and D. Xu, "Software-based realtime recovery from sensor attacks on robotic vehicles," in *Proc. Int. Symp. Res. Attacks, Intrusions Defenses*, 2020, pp. 349–364.
- [14] X. Han et al., "A unified anomaly detection methodology for lane-following of autonomous driving systems," in *Proc. IEEE Trans. Parallel. Distrib. Syst.*, 2021, pp. 836–844.
- [15] P. Yanik, G. Ford, and W. McDaniel, "An introduction and literature review of fuzzy logic applications for robot motion planning," in *Proc. ASEE Southeast Sect. Conf.*, 2010, pp. 1–10.
- [16] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, and J. Pan, "Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2018, pp. 6252–6259.
- [17] K. Wu, H. Wang, M. A. Esfahani, and S. Yuan, "Learn to navigate autonomously through deep reinforcement learning," *IEEE Trans. Ind. Electron.*, vol. 69, no. 5, pp. 5342–5352, May 2022.
- [18] S. H. Semnani, H. Liu, M. Everett, A. de Ruiter, and J. P. How, "Multi-agent motion planning for dense and dynamic environments via deep reinforcement learning," *IEEE Robot. Automat. Lett.*, vol. 5, no. 2, pp. 3221–3226, Feb. 2020.

- [19] M. Everett, Y. F. Chen, and J. P. How, "Motion planning among dynamic, decision-making agents with deep reinforcement learning," in *Proc. IEEE Int. Conf. Intell. Robots Syst.*, 2018, pp. 3052–3059.
- [20] P. Dash, G. Li, Z. Chen, M. Karimibiuki, and K. Pattabiraman, "PID-Piper: Recovering robotic vehicles from physical attacks," in *Proc. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2021, pp. 26–38.
- [21] D. J. Yeong et al., "Sensor and sensor fusion technology in autonomous vehicles: A review," *Sensors*, vol. 21, no. 6, 2021, Art. no. 2140.
- [22] F. Fei, Z. Tu, D. Xu, and X. Deng, "Learn-to-recover: Retrofitting UAVs with reinforcement learning-assisted flight control under cyber-physical attacks," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2020, pp. 7358–7364.
- [23] E. Caldarelli, A. Colomé, and C. Torras, "Perturbation-based stiffness inference in variable impedance control," *IEEE Robot. Automat. Lett.*, vol. 7, no. 4, pp. 8823–8830, Oct. 2022.
- [24] I. Goodfellow et al., "Generative adversarial networks," *Commun. ACM*, vol. 63, no. 11, pp. 139–144, 2020.
- [25] A. Jabbar, X. Li, and B. Omar, "A survey on generative adversarial networks: Variants, applications, and training," *ACM Comput. Surv.*, vol. 54, no. 8, pp. 1–49, 2021.
- [26] C. Suliman, C. Cruceru, and F. Moldoveanu, "Mobile robot position estimation using the Kalman filter," *Acta Marisiensis. Seria Technologica*, vol. 6, p. 75, 2009.